PROJECT ON

ONLINE PAYMENT FRAUD DETECTION

BY- ABHISHEK RAWAT

PREFACE

In an increasingly digital world, the convenience of online transactions has transformed the way we conduct business and manage our finances. However, this shift has also given rise to a significant challenge: online payment fraud. As e-commerce continues to expand, so does the sophistication of fraudulent activities, posing risks not only to consumers but also to businesses and financial institutions. The need for effective fraud detection mechanisms has never been more critical.

This project on "Online Payment Fraud Detection" aims to address this pressing issue by leveraging advanced data analysis techniques and machine learning algorithms. Utilizing a comprehensive dataset that encompasses various transaction attributes, our goal is to develop a robust model capable of identifying fraudulent activities in real-time. By analysing patterns and anomalies within the data, we seek to enhance the security of online payment systems and protect stakeholders from potential losses.

The dataset used in this project is rich in features, including transaction amounts, user behaviour, and geographical information, which allows for a nuanced understanding of the factors contributing to fraud. Through rigorous data preprocessing, feature engineering, and model evaluation, we aim to create a solution that not only detects fraud effectively but also minimizes false positives, ensuring a seamless experience for legitimate users.

We acknowledge the collaborative efforts of researchers, data scientists, and industry experts who have contributed to the field of fraud detection. We hope that our findings will not only advance the understanding of online payment fraud but also provide practical insights for businesses seeking to enhance their security measures.

This project represents a step towards a safer online payment ecosystem. We invite others to explore our methodologies, findings, and recommendations, and to add any insight if they have to fight against online payment fraud.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who contributed to the successful completion of the project titled "Online Payment Fraud Detection."

First and foremost, we would like to thank Kaggle for providing the dataset that served as the foundation for our analysis. The availability of diverse and comprehensive datasets on Kaggle significantly enhanced our ability to develop and test our fraud detection algorithms.

We are also deeply grateful to Dr. R. Nageswara Rao, whose book "Machine Learning in Data Science using Python" was instrumental in guiding us through the coding and implementation processes. The insights and methodologies presented in this book provided us with a solid framework for understanding machine learning concepts and applying them effectively in our project.

Additionally, we would like to acknowledge the support and resources provided by Intellipaat Institute. Their training and guidance were invaluable in equipping us with the necessary skills and knowledge to tackle the challenges of this project.

Finally, we would like to thank our peers and mentors for their encouragement and constructive feedback throughout the project. Their support has been crucial in refining our approach and enhancing the quality of our work.

ABOUT DATASET

To identify online payment fraud with machine learning, we need to train a machine learning model for classifying fraudulent and non-fraudulent payments. For this, we need a dataset containing information about online payment fraud, so that we can understand what type of transactions lead to fraud. For this task, we have collected a dataset from Kaggle, which contains historical information about fraudulent transactions which can be used to detect fraud in online payments. Below are all the columns from the dataset we are using here:

step: represents a unit of time where 1 step equals 1 hour.
type: type of online transaction.
amount: the amount of the transaction.
nameOrig: customer starting the transaction.
oldbalanceOrg: balance before the transaction.
newbalanceOrig: balance after the transaction.
nameDest: recipient of the transaction.
oldbalanceDest: initial balance of recipient before the transaction.
newbalanceDest: the new balance of recipient after the transaction.
isFraud: fraud transaction.

	A	B	C	D	E	F	G	H	I	
1	step	type	amount	nameOrig	oldbalance	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud
2	1	PAYMENT	9839.64	C1231006	170136	160296.36	M1979787155	0	0	
3	1	PAYMENT	1864.28	C1666544	21249	19384.72	M2044282225	0	0	
4	1	TRANSFER	181	C1305486	181	0	C553264065	0	0	
5	1	CASH_OU	181	C8400836	181	0	C38997010	21182	0	
6	1	PAYMENT	11668.14	C2048537	41554	29885.86	M1230701703	0	0	
7	1	PAYMENT	7817.71	C9004563	53860	46042.29	M573487274	0	0	
8	1	PAYMENT	7107.77	C1549888	183195	176087.23	M408069119	0	0	
9	1	PAYMENT	7861.64	C1912850	176087.2	168225.59	M633326333	0	0	
10	1	PAYMENT	4024.36	C1265012	2671	0	M1176932104	0	0	
11	1	DEBIT	5337.77	C7124101	41720	36382.23	C195600860	41898	40348.79	
12	1	DEBIT	9644.94	C1900366	4465	0	C997608398	10845	157982.12	
13	1	PAYMENT	3099.97	C2491775	20771	17671.03	M2096539129	0	0	
14	1	PAYMENT	2560.74	C1648232	5070	2509.26	M972865270	0	0	
15	1	PAYMENT	11633.76	C1716932	10127	0	M801569151	0	0	
16	1	PAYMENT	4098.78	C1026483	503264	499165.22	M1635378213	0	0	
17	1	CASH_OU	229133.9	C9050804	15325	0	C476402209	5083	51513.44	
18	1	PAYMENT	1563.82	C7617507	450	0	M1731217984	0	0	
19	1	PAYMENT	1157.86	C1237762	21156	19998.14	M1877062907	0	0	
20	1	PAYMENT	671.64	C2033524	15123	14451.36	M473053293	0	0	
21	1	TRANSFER	215310.3	C1670993	705	0	C1100439041	22425	0	
22	1	PAYMENT	1373.43	C2080460	13854	12480.57	M1344519051	0	0	
23	1	DEBIT	9302.79	C1566511	11299	1996.21	C1973538135	29832	16896.7	
24	1	DEBIT	1065.41	C1959239	1817	751.59	C515132998	10330	0	
25	1	PAYMENT	3876.41	C5043364	67852	63975.59	M1404932042	0	0	
26	1	TRANSFER	311685.9	C1984094	10835	0	C932583850	6267	2719172.89	
27	1	PAYMENT	6061.13	C1043358	443	0	M1558079303	0	0	

Now, as we have discussed the dataset we are using for the online payment fraud detection, furthermore, we'll explain how we can use Machine Learning to detect online payment fraud using python.

PERFORMING EDA(EXPLORATORY DATA ANALYSIS)

Exploratory Data Analysis (EDA) is crucial for understanding the structure and patterns within the "online payment fraud detection" dataset. It helps identify anomalies, relationships, and trends in the data, which are essential for building effective fraud detection models.

Importance of Exploratory Data Analysis (EDA)

- **Understanding Data Distribution**: EDA allows for the examination of the distribution of key variables, such as the target variable **isFraud**, which helps in identifying class imbalances. In this dataset, only a small percentage of transactions are fraudulent, indicating a need for specialized techniques to handle imbalanced data.
- **Identifying Relationships**: By analyzing correlations between features like **amount**, **oldbalanceOrg**, and **newbalanceDest**, EDA can reveal important relationships that may influence fraud detection. Understanding these relationships can guide feature selection and engineering.
- **Detecting Anomalies**: EDA helps in spotting outliers or unusual patterns in transaction amounts or balances, which could indicate fraudulent activity. For instance, examining the **amount** column can highlight transactions that are significantly higher than typical values.
- **Transaction Type Analysis**: Analyzing the **type** column can provide insights into which transaction types are more prone to fraud. This can inform targeted strategies for monitoring specific transaction types, such as **TRANSFER** and **CASH_OUT**.
- **Temporal Patterns**: By exploring the **step** column, which represents time, EDA can uncover trends related to the timing of transactions. For example, identifying peak hours for fraudulent transactions can help in developing time-sensitive fraud detection strategies.
- **Feature Engineering Opportunities**: EDA can reveal opportunities for creating new features, such as deriving the **hour** from the **step** variable, which may enhance model performance by capturing time-related patterns in fraud occurrences.
- **Visual Insights**: Utilizing visualizations, such as histograms, box plots, and heatmaps, can provide intuitive insights into the data, making it easier to communicate findings to stakeholders and guide further analysis.

Now let's load the data set and perform the comprehensive Exploratory Data Analysis including:

1) Basic statistics

2) Data types and missing values

3) Distribution of key variables

4) Outlier detection

5) Fraud vs Non-Fraud comparisons

EXPLORATORY DATA ANALYSIS

```
[7] import numpy as np
    import pandas as pd
    import matplotlib.pyplot as plt
    import seaborn as sns
```

```
[9] df = pd.read_excel("/content/onlinefraud.csv.xlsx")
```

```
[ ] Start coding or generate with AI.
```

```
#Basic dataset statistics
eda_summary = {
    "shape": df.shape,
    "data_types": df.dtypes,
    "missing_values": df.isnull().sum(),
    "summary_stats": df.describe(),
    "fraud_counts": df["isFraud"].value_counts(),
    "fraud_percent": df["isFraud"].value_counts(normalize=True) * 100,
    "transaction_types": df["type"].value_counts()
}
```

```
# Display each part of the summary
print("📄 Shape of the dataset:")
print(eda_summary["shape"])
print("\n📦 Data types:")
print(eda_summary["data_types"])
print("\n🔍 Missing values:")
print(eda_summary["missing_values"])
print("\n📊 Summary statistics (numerical columns):")
print(eda_summary["summary_stats"])
print("\n🚨 Fraud counts:")
```

Commands + Code + Text Run all

```python
print("\n🔄 Transaction type distribution:")
print(eda_summary["transaction_types"])
```

```
isFlaggedFraud      int64
dtype: object

🔍 Missing values:
step              0
type              0
amount            0
nameOrig          0
oldbalanceOrg     0
newbalanceOrig    0
nameDest          0
oldbalanceDest    0
newbalanceDest    0
isFraud           0
isFlaggedFraud    0
dtype: int64

📊 Summary statistics (numerical columns):
                step        amount  oldbalanceOrg  newbalanceOrig  \
count  1.048575e+06  1.048575e+06   1.048575e+06    1.048575e+06
mean   2.696617e+01  1.586670e+05   8.740055e+05    8.938049e+05
std    1.562325e+01  2.649409e+05   2.971725e+06    3.008246e+06
min    1.000000e+00  1.000000e-01   0.000000e+00    0.000000e+00
25%    1.500000e+01  1.214907e+04   0.000000e+00    0.000000e+00
50%    2.000000e+01  7.634333e+04   1.600200e+04    0.000000e+00
75%    3.900000e+01  2.137619e+05   1.366420e+05    1.746000e+05
max    9.500000e+01  1.000000e+07   3.893942e+07    3.894623e+07

       oldbalanceDest  newbalanceDest       isFraud  isFlaggedFraud
count    1.048575e+06    1.048575e+06  1.048575e+06       1048575.0
mean     9.781600e+05    1.114193e+06  1.089097e-03             0.0
std      2.296779e+06    2.416554e+06  3.298351e-02             0.0
min      0.000000e+00    0.000000e+00  0.000000e+00             0.0
25%      0.000000e+00    0.000000e+00  0.000000e+00             0.0
50%      1.263772e+05    2.182604e+05  0.000000e+00             0.0
75%      9.159235e+05    1.149898e+06  0.000000e+00             0.0
```

Variables Terminal 11:11 AM Python 3

colab.research.google.com/drive/1OtA_FvWVGV426sQak2-s3HbKvShiNyVH#scrollTo=2QteCcVYTi8G

Commands + Code + Text Run all

```
🚨 Fraud counts:
isFraud
0    1047433
1       1142
Name: count, dtype: int64

📈 Fraud percentage:
isFraud
0    99.89109
1     0.10891
Name: proportion, dtype: float64

💳 Transaction type distribution:
type
CASH_OUT    373641
PAYMENT     353873
CASH_IN     227130
TRANSFER     86753
DEBIT         7178
Name: count, dtype: int64
```

Variables Terminal 11:11 AM Python 3

Here are the few details from the above data

🚨 **Fraud counts:**

isFraud

0 1047433

1 1142

📈 **Fraud percentage:**

isFraud

0 99.89109

1 0.10891

💳 **Transaction type distribution:**

type

CASH_OUT 373641

PAYMENT 353873

CASH_IN 227130

TRANSFER 86753

DEBIT 7178

📊 **Fraud Percentage Calculation**

- Non-Fraud (isFraud = 0): 1,047,433
- Fraud (isFraud = 1): 1,142

  Percentage of Fraud: Fraud Percentage=(1142+1,047,4331142)×100≈0.109%

The dataset shows that only **0.109% of transactions are fraudulent.** To avoid those 0.109% fraudulent transactions, careful handling in model development, such as resampling, anomaly detection, or weighted algorithms is required, to ensure that fraud cases are not ignored during prediction.

Now, moving with the detection of outliers:

colab.research.google.com/drive/1OtA_FvWVGV426sQak2-s3HbKvShiNyVH#scrollTo=juXjARbbcS8E

Commands + Code + Text Run all Reconnect

```python
# Detecting outliers via IQR
def detect_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = data[(data[column] < lower_bound) | (data[column] > upper_bound)]
    return outliers, lower_bound, upper_bound
```

```python
# Applying to relevant numeric columns
for col in ['amount', 'oldbalanceOrg', 'newbalanceOrig', 'oldbalanceDest', 'newbalanceDest']:
    outliers, low, high = detect_outliers_iqr(df, col)
    print(f"{col}: {len(outliers)} outliers between {low} and {high}")
```

```
amount: 53088 outliers between -290270.17250000004 and 516181.12750000006
oldbalanceOrg: 181877 outliers between -204963.03000000003 and 341605.05000000005
newbalanceOrig: 170244 outliers between -261899.985 and 436499.975
oldbalanceDest: 125403 outliers between -1373885.2125 and 2289808.6875
newbalanceDest: 114557 outliers between -1724711.2649999997 and 2874518.7749999994
```

```python
# Fraud analysis by type
fraud_by_type = df[df['isFraud'] == 1]['type'].value_counts()
fraud_by_type_percent = fraud_by_type / df['type'].value_counts() * 100
```

```python
# Print results
print("🔍 Fraud Count by Transaction Type:")
print(fraud_by_type)

print("\n📊 Fraud Percentage by Transaction Type:")
print(fraud_by_type_percent)
```

Variables Terminal 11:52 AM

colab.research.google.com/drive/1OtA_FvWVGV426sQak2-s3HbKvShiNyVH#scrollTo=juXjARbbcS8E

Commands + Code + Text Run all Reconnect

```
🔍 Fraud Count by Transaction Type:
type
CASH_OUT    578
TRANSFER    564
Name: count, dtype: int64

📊 Fraud Percentage by Transaction Type:
type
CASH_IN          NaN
CASH_OUT    0.154694
DEBIT            NaN
PAYMENT          NaN
TRANSFER    0.650122
Name: count, dtype: float64
```

+ Code + Text

Variables Terminal 11:52 AM

Applying fraud analysis by type to relevant columns and detection of outliers

🔍 **Fraud Count by Transaction Type:**

type

CASH_OUT 578

TRANSFER 564

📊 **Fraud Percentage by Transaction Type:**

type

CASH_IN NaN

CASH_OUT 0.154694

DEBIT NaN

PAYMENT NaN

TRANSFER 0.650122

OUTLIER DETECTION INFERENCE

1) **Large number of outliers detected** — particularly in oldbalanceOrg, oldbalanceDest, and newbalanceDest. This indicates **a high degree of variability in transaction sizes and balances**, which is typical in financial datasets.

2) These outliers are likely **high-value transactions**, which are **more likely to be fraudulent**. This supports the idea that fraudsters target larger transfers or withdrawals to maximize gain.

FRAUD ANALYSIS BY TRANSACTION TYPE

Transaction Type	Fraud Count	Fraud %
CASH_OUT	**578**	**0.15%**
TRANSFER	**564**	**0.65%**
Others	**0**	**0.00%**

A) This shows that only **Transfer** and **Cash out** transactions are fraudulent.
B) All frauds occur in money moving transactions.
C) No frauds were found in **Payment** , **Cash In** or **Debit**.
D) Transfer has the highest fraud percentage (0.65% approx.)

We have framed 12 questions relevant to the "Online Payment Fraud Detection" project that can be explored using the provided dataset. These questions can guide both marketing and research efforts, and they can be analyzed using Python and data visualization techniques. The questions framed for the "Online Payment Fraud Detection" project are designed to explore various dimensions of transaction data, focusing on identifying patterns, trends, and anomalies that could indicate fraudulent behavior. Each question serves a specific purpose, contributing to a comprehensive understanding of the dataset and enhancing the effectiveness of fraud detection mechanisms.

1. **Fraud Rate Analysis**: What is the overall percentage of transactions that are flagged as fraud in the dataset?

   This question aims to quantify the prevalence of fraud within the dataset. Understanding the overall fraud rate helps establish a baseline for evaluating the effectiveness of fraud detection systems and can inform stakeholders about the scale of the issue.

   As we have earlier calculated the Fraud Percentage of the dataset we are using

   Fraud Percentage Calculation

   • Non-Fraud (isFraud = 0): 1,047,433

   • Fraud (isFraud = 1): 1,142

   Percentage of Fraud:

   Fraud Percentage=(1142+1,047,4331142)×100**≈0.109%**

   Based on the **overall fraud percentage of ≈0.109%**, here are key insights and inferences we have drawn:

   1) **Only ~0.11%** of all transactions are fraudulent — this makes fraud events **very rare**.
   2) Despite the rarity, the **financial and reputational impact of fraud is high**, so detection remains important.
   3) Even though frauds are rare, **missing a fraud (false negative)** could result in significant loss.

Summary:

The fraud rate is low**≈0.109%**. While fraud is uncommon, the financial risk is substantial, making it critical to detect even a small number of fraudulent transactions effectively**.**

2) Transaction Amount Distribution: By comparing the distribution of transaction amounts between legitimate and fraudulent transactions, we can identify thresholds or patterns that are indicative of fraud. Visualizing this data through box plots can highlight outliers and trends that warrant further investigation.

By comparing the distribution of transaction amounts between legitimate and fraudulent transactions, we can identify thresholds or patterns that are indicative of fraud. Visualizing this data through box plots can highlight outliers and trends that warrant further investigation

How it was done

We have already uploaded the dataset in **Google Colab** for all the analysis and performing all the functions.

Here is the coding we have applied to get the desired outcome/result of the given question.

```
#importing of the libraries

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

#performing the coding to get the box plot

plt.figure(figsize=(8, 4))

sns.boxplot(x='isFraud', y='amount', data=df)

plt.yscale('log')  # Helps visualize extreme values

plt.title("Transaction Amount Distribution by Fraud Status")

plt.xlabel("isFraud (0 = Legitimate, 1 = Fraudulent)")

plt.ylabel("Transaction Amount (log scale)")

plt.grid(True)

plt.tight_layout()

plt.show()
```

Formula behind the box plot

Q1 (25th percentile)

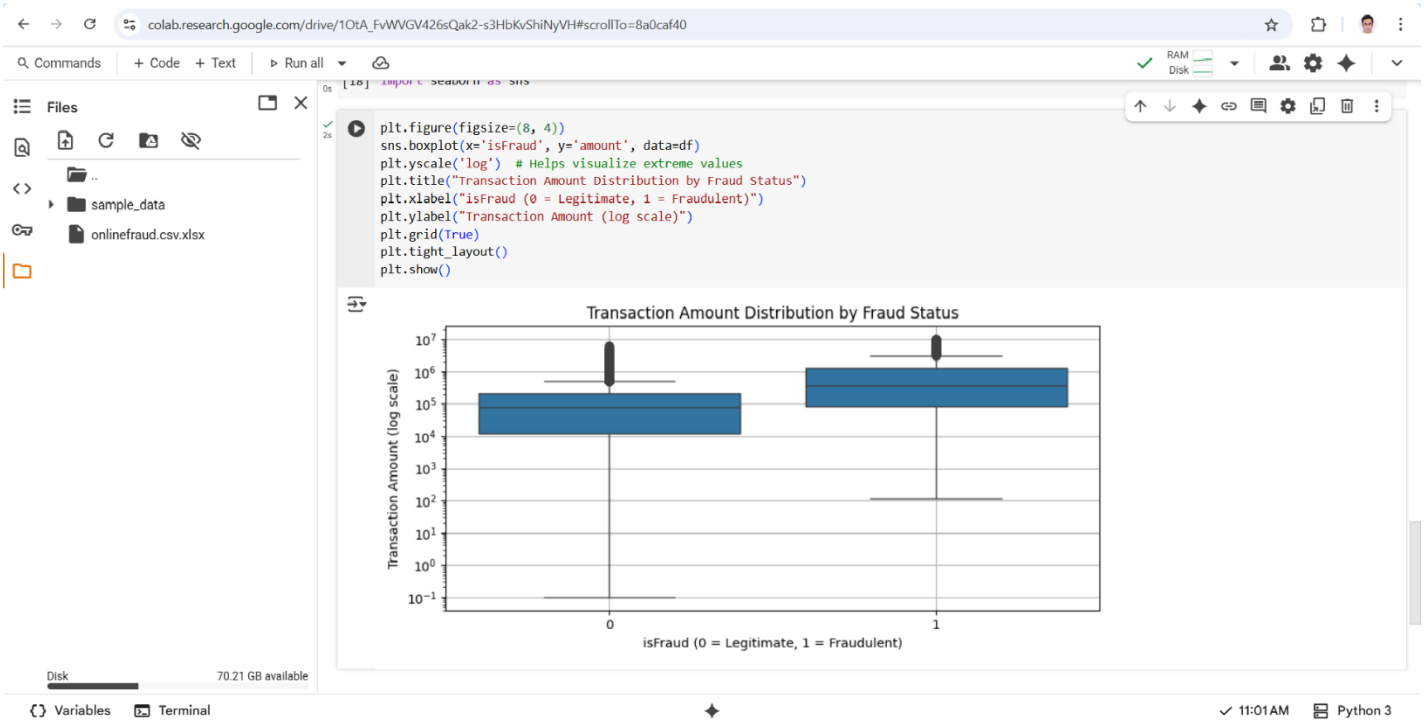
Q2 (median / 50th percentile)

Q3 (75th percentile)

IQR = Q3 - Q1 Outliers are defined as

Lower bound=Q1−1.5×IQR

Upper bound=Q3+1.5×IQR



Summary:

1) Fraudulent transactions are higher in value and show less variation, focusing around specific high-value targets.
2) The interquartile range **(IQR)** for fraud is **narrower**, meaning most fraud amounts are **clustered** within a specific high range. This may be an indication that fraudsters are using a **'preset' or safe amount,** to avoid detection.
3) The IQR is wide and the spread of the amounts is highly variable, showing normal banking behaviour with a broad range of banking transactions.
4) **Fraudulent transactions tend to be high-value and less varied**, while legitimate ones have a broader range of amounts. This insight can guide rule-based fraud detection and support machine learning models by emphasizing amount-related features and outlier detection.

3) Balance Impact: What is the average old and new balance for both origin and destination accounts in fraudulent transactions compared to non-fraudulent transactions?

Analyzing the old and new balances of accounts involved in fraudulent transactions can reveal behavioral patterns. Significant differences in balances may indicate unusual activity, helping to refine fraud detection criteria.

How it was done

Here is the coding we have applied to get the desired outcome/result of the given question.

```
# Calculate average balances for fraudulent transactions
fraudulent = df[df['isFraud'] == 1]
avg_fraudulent = {
    'origin_old_balance': fraudulent['oldbalanceOrg'].mean(),
    'origin_new_balance': fraudulent['newbalanceOrig'].mean(),
    'destination_old_balance': fraudulent['oldbalanceDest'].mean(),
    'destination_new_balance': fraudulent['newbalanceDest'].mean()
# Calculate average balances for non-fraudulent transactions
non_fraudulent = df[df['isFraud'] == 0]
avg_non_fraudulent = {
    'origin_old_balance': non_fraudulent['oldbalanceOrg'].mean(),
    'origin_new_balance': non_fraudulent['newbalanceOrig'].mean(),
    'destination_old_balance': non_fraudulent['oldbalanceDest'].mean(),
    'destination_new_balance': non_fraudulent['newbalanceDest'].mean()
}
# Combine results
average_balances = {
    'fraudulent': avg_fraudulent,
    'non_fraudulent': avg_non_fraudulent
}
print(average_balances)
```

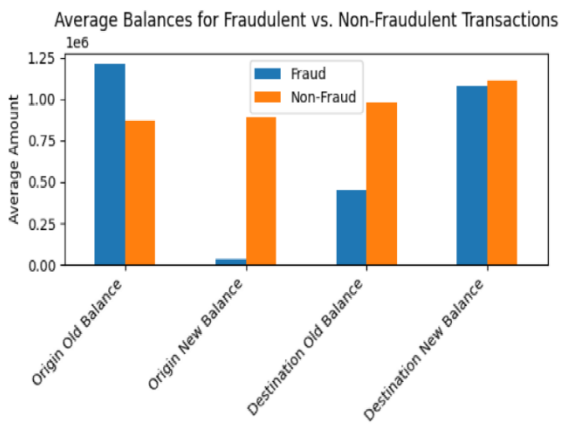
Summary of Averages

Metric	Fraudulent Transactions	Non-Fraudulent Transactions
Origin Old Balance	**₹1,218,656**	**₹873,630**
Origin New Balance	**₹33,944**	**₹894,742**
Destination Old Balance	**₹452,804**	**₹978,733**
Destination New Balance	**₹1,077,944**	**₹1,114,233**

We can also visualize the data obtained in a bar graph

```
df_plot = pd.DataFrame(df).T

# Plotting the average balances

df_plot.plot(kind='bar', figsize=(6, 4))

plt.title('Average Balances for Fraudulent vs. Non-Fraudulent Transactions')

plt.ylabel('Average Amount')

plt.xticks(rotation=45, ha='right')

plt.tight_layout()

plt.show()
```

```python
# Plotting the average balances
df_plot.plot(kind='bar', figsize=(6, 4))
plt.title('Average Balances for Fraudulent vs. Non-Fraudulent Transactions')
plt.ylabel('Average Amount')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



Variables Terminal ✓ 6:32 PM Python 3

Summary:

1) In Fraudulent Transactions the origin account's balance drops from **1,218,656 to 33,944** which is a significant drop indicating that fraudsters are likely emptying the account in one go.

2) Non fraudulent transactions are high and steady.

3) Fraudulent transactions often involve draining high-balance origin accounts and transferring large sums to destination accounts in a single transaction. These patterns can be used to build fraud detection rules (e.g., flagging sudden balance drops or unusually large amount of transfer increases).

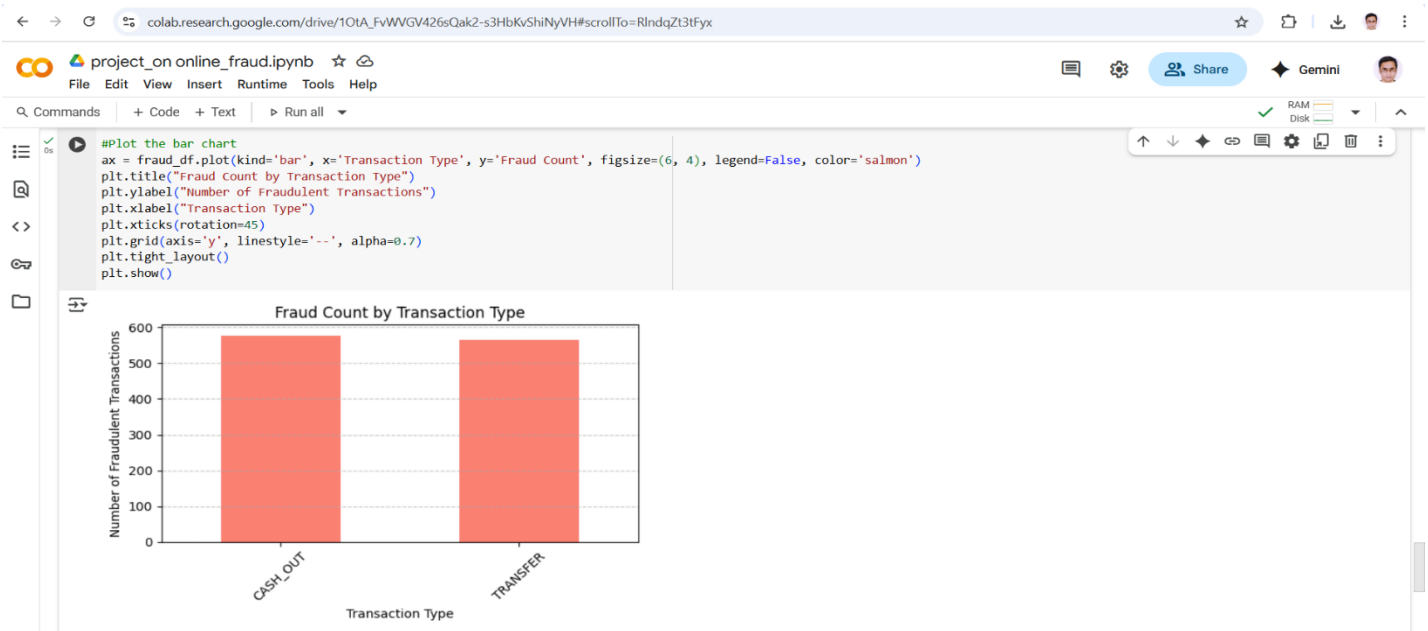
4) Fraud by Transaction Type: Which type of transactions (e.g., credit, debit) are most commonly associated with fraud? Can we create a bar chart to illustrate this?

Understanding which transaction types are most frequently associated with fraud can guide targeted interventions. A bar chart visualization can effectively communicate this information to stakeholders, allowing for focused risk management strategies.

How it was done

Here is the coding we have applied to get the desired outcome/result of the given question.

```
#Convert the Series to a DataFrame and reset the index

fraud_df = pd.DataFrame(fraud_by_type).reset_index()

fraud_df.columns = ['Transaction Type', 'Fraud Count']

#Plot the bar chart

ax = fraud_df.plot(kind='bar', x='Transaction Type', y='Fraud Count', figsize=(6, 4), legend=False, color='salmon')

plt.title("Fraud Count by Transaction Type")

plt.ylabel("Number of Fraudulent Transactions")

plt.xlabel("Transaction Type")

plt.xticks(rotation=45)

plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()

plt.show()
```



Summary

1) CASH_OUT transactions have the highest count of fraud cases (578). TRANSFER transactions are the second most common in fraud cases (564). All other types — CASH_IN, PAYMENT, and DEBIT — have zero fraudulent transactions.

2) Fraud detection and prevention efforts should focus primarily on monitoring TRANSFER and CASH_OUT transactions.

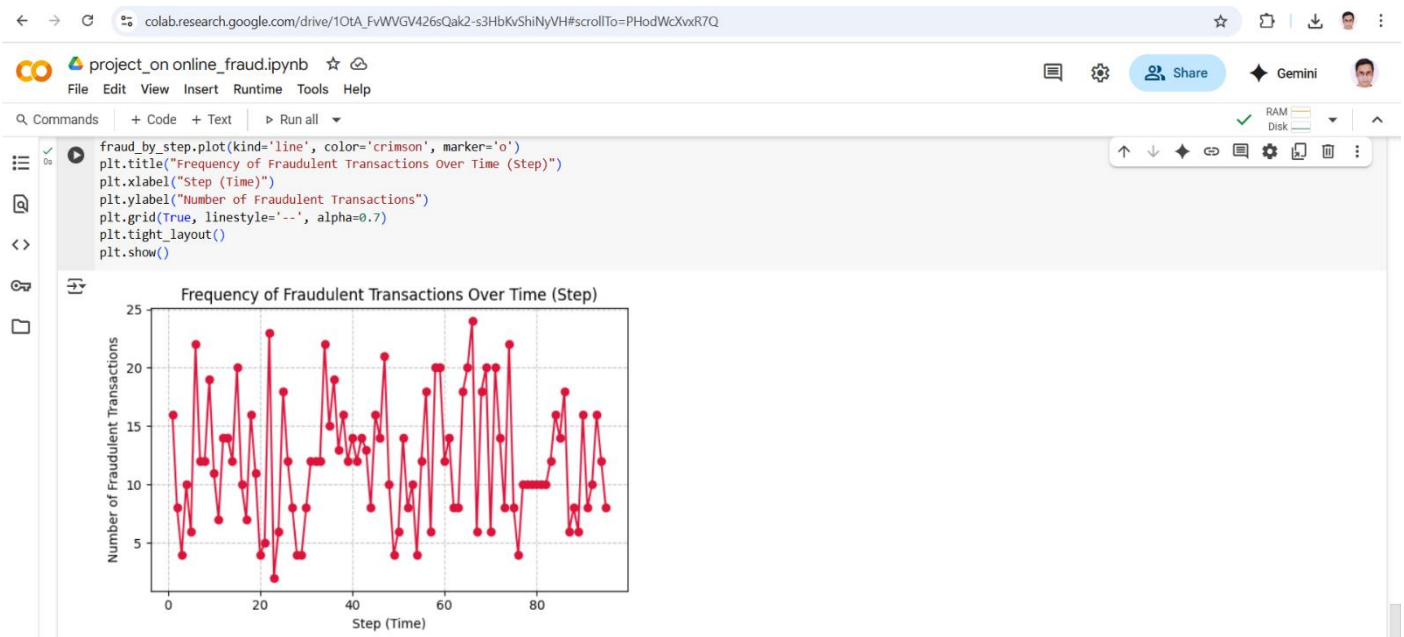
5) Time Series Analysis: How does the frequency of fraudulent transactions change over time? Can we visualize this trend using a time series plot?

Investigating how the frequency of fraudulent transactions changes over time can uncover seasonal trends or emerging threats. A time series plot can help in forecasting future fraud risks and adjusting detection strategies accordingly

How it was done

Here is the coding we have applied to get the desired outcome/result of the given question.

```
#Filter only fraud transactions

fraud_df = df[df['isFraud'] == 1]

fraud_by_step = fraud_df.groupby('step').size()

#Plotting the time series

plt.figure(figsize=(6, 4))

fraud_by_step.plot(kind='line', color='crimson', marker='o')

plt.title("Frequency of Fraudulent Transactions Over Time (Step)")

plt.xlabel("Step (Time)")

plt.ylabel("Number of Fraudulent Transactions")

plt.grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()

plt.show()
```



Summary:

1) The number of fraudulent transactions fluctuates over time, as there are multiple spikes.

2) There is no clear increasing or decreasing trend, however a few clusters show higher fraud density around step 10–20, 40–45, 65–70.

6) Account Behavior: What patterns can be observed in the old and new balances of accounts involved in fraudulent transactions? Are there any significant differences compared to non-fraudulent transactions?

This question seeks to identify behavioral anomalies in accounts involved in fraud. By comparing balances, we can detect unusual patterns that may indicate fraudulent activity, leading to more proactive monitoring of at-risk accounts.

How it was done

We have used a bar graph to visualize the following result from the answer

Here is the coding we have applied to get the desired outcome/result of the given question.

```
# Filter fraudulent and non-fraudulent transactions

fraudulent = df[df['isFraud'] == 1]

non_fraudulent = df[df['isFraud'] == 0]

# Calculate old and new balances for both types

fraudulent_balances = fraudulent[['oldbalanceOrg', 'newbalanceOrig', 'oldbalanceDest',
'newbalanceDest']]

non_fraudulent_balances = non_fraudulent[['oldbalanceOrg', 'newbalanceOrig', 'oldbalanceDest',
'newbalanceDest']]

mean_fraudulent = fraudulent_balances.mean()

mean_non_fraudulent = non_fraudulent_balances.mean()

# Display the results

print("Mean Balances for Fraudulent Transactions:")

print(mean_fraudulent)

print("\nMean Balances for Non-Fraudulent Transactions:")

print(mean_non_fraudulent)
```

Mean Balances for Fraudulent Transactions:

oldbalanceOrg 1.218656e+06

newbalanceOrig 3.394432e+04

oldbalanceDest 4.528040e+05

newbalanceDest 1.077944e+06

Mean Balances for Non-Fraudulent Transactions:

oldbalanceOrg 8.736298e+05

newbalanceOrig 8.947423e+05

oldbalanceDest 9.787328e+05

newbalanceDest 1.114233e+06

```
# Visualize the differences
import matplotlib.pyplot as plt
labels = ['Old Balance Org', 'New Balance Orig', 'Old Balance Dest', 'New Balance Dest']
fraudulent_means = mean_fraudulent.values
non_fraudulent_means = mean_non_fraudulent.values
x = range(len(labels))
plt.bar(x, fraudulent_means, width=0.4, label='Fraudulent', color='red', align='center')
plt.bar([p + 0.4 for p in x], non_fraudulent_means, width=0.4, label='Non-Fraudulent', color='green',
align='center')
plt.xlabel('Transaction Balances')
plt.ylabel('Mean Values')
plt.title('Comparison of Balances in Fraudulent vs Non-Fraudulent Transactions')
plt.xticks([p + 0.2 for p in x], labels)
plt.legend()
plt.show()
```

project_on online_fraud.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share Gemini

Files

..

sample_data

onlinefraud.csv.xlsx

Disk 70.18 GB available

[26]
```
print(mean_non_fraudulent)
```

```
Mean Balances for Fraudulent Transactions:
oldbalanceOrg     1.218656e+06
newbalanceOrig    3.394432e+04
oldbalanceDest    4.528040e+05
newbalanceDest    1.077944e+06
dtype: float64

Mean Balances for Non-Fraudulent Transactions:
oldbalanceOrg     8.736298e+05
newbalanceOrig    8.947423e+05
oldbalanceDest    9.787328e+05
newbalanceDest    1.114233e+06
dtype: float64
```

```
# Visualize the differences
import matplotlib.pyplot as plt

labels = ['Old Balance Org', 'New Balance Orig', 'Old Balance Dest', 'New Balance Dest']
fraudulent_means = mean_fraudulent.values
non_fraudulent_means = mean_non_fraudulent.values

x = range(len(labels))
plt.bar(x, fraudulent_means, width=0.4, label='Fraudulent', color='red', align='center')
plt.bar([p + 0.4 for p in x], non_fraudulent_means, width=0.4, label='Non-Fraudulent', color='green', align='center')

plt.xlabel('Transaction Balances')
plt.ylabel('Mean Values')
plt.title('Comparison of Balances in Fraudulent vs Non-Fraudulent Transactions')
plt.xticks([p + 0.2 for p in x], labels)
plt.legend()
```

Variables Terminal 7:04 PM Python 3

colab.research.google.com/drive/1OtA_FvWVGV426sQak2-s3HbKvShiNyVH?pli=1#scrollTo=g3eXYus_ckrb

project_on online_fraud.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share Gemini

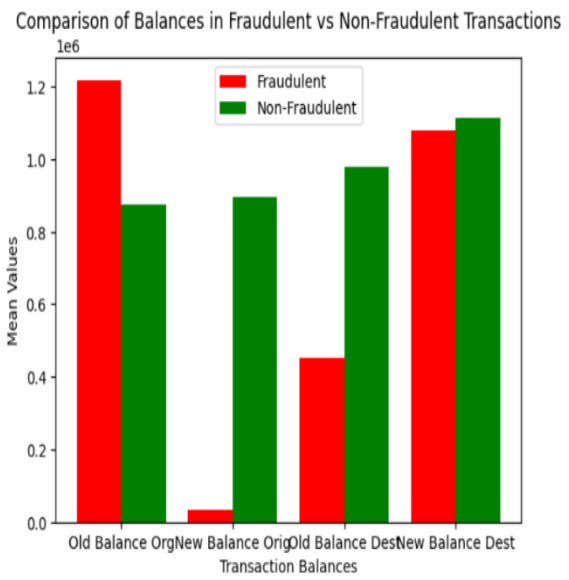
Files

..

sample_data

onlinefraud.csv.xlsx

```
plt.legend()
plt.show()
```



Summary:

1) In fraud cases, the new balance drops very sharply, while in non-fraud cases, it remains almost unchanged. This pattern suggests complete fund depletion in fraudulent withdrawals (e.g., CASH_OUT or TRANSFER), which is uncommon in legitimate transactions.

2) In non-fraud transactions, destination accounts start with higher balances.

3) In New Balance Destination Account, both types show an increase, but **fraudulent destination balances jump sharply** possibly due to large single transfers.

7) Destination Account Analysis: How many unique destination accounts are involved in fraudulent transactions, and what is the average transaction amount for these accounts?

Analyzing unique destination accounts involved in fraud can help identify potential networks of fraudulent activity. Understanding the average transaction amount for these accounts can inform risk assessments and fraud prevention measures.

How it was done

Here is the coding we have applied to get the desired outcome/result of the given question.

```
# Filter fraudulent transactions

fraudulent_transactions = df[df['isFraud'] == 1]

# Calculate unique destination accounts and average transaction amount

unique_dest_accounts = fraudulent_transactions['nameDest'].nunique()

average_transaction_amount = fraudulent_transactions['amount'].mean()

print(f"Unique destination accounts involved in fraudulent transactions: {unique_dest_accounts}")

print(f"Average transaction amount for these accounts: {average_transaction_amount}")
```

The answers came out to be

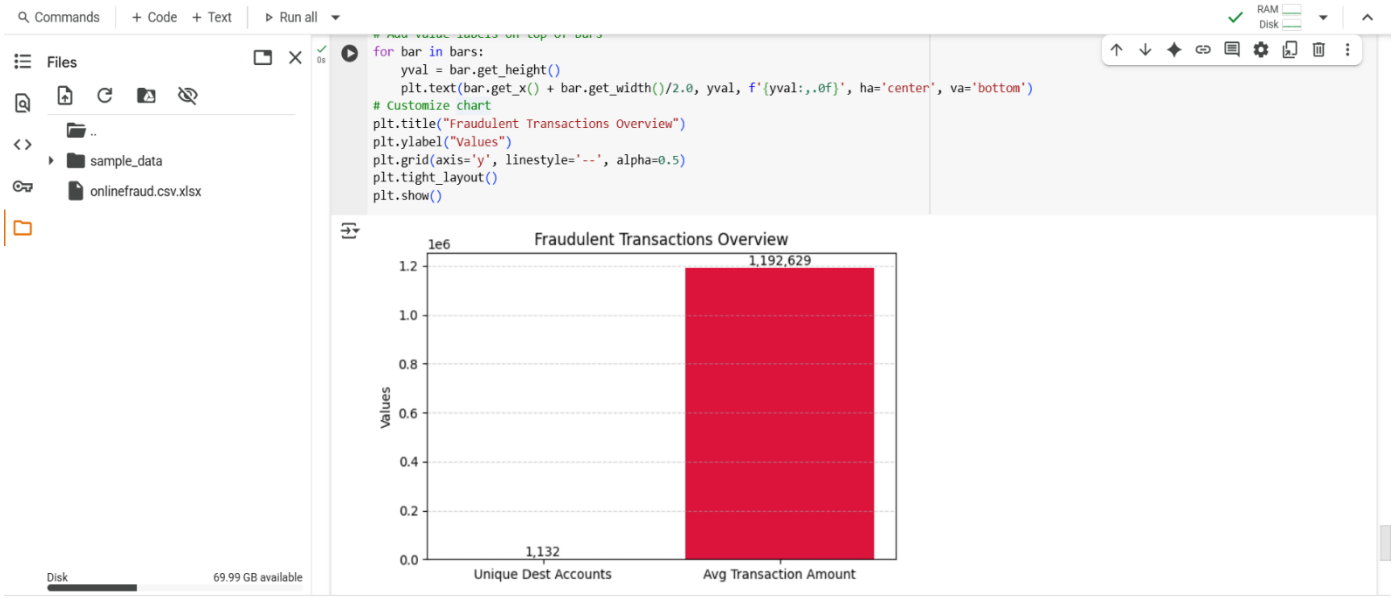
Unique destination accounts involved in fraudulent transactions: 1132

Average transaction amount for these accounts: 1192628.9320753065

To show them more clearly, we have used **bar chart,** the coding of which is as follows:

```
import matplotlib.pyplot as plt

# Data to plot

labels = ['Unique Dest Accounts', 'Avg Transaction Amount']

values = [unique_dest_accounts, average_transaction_amount]

# Create bar chart

plt.figure(figsize=(6, 4))

bars = plt.bar(labels, values, color=['royalblue', 'crimson'])

# Add value labels on top of bars

for bar in bars:

    yval = bar.get_height()

    plt.text(bar.get_x() + bar.get_width()/2.0, yval, f'{yval:,.0f}', ha='center', va='bottom')

# Customize chart

plt.title("Fraudulent Transactions Overview")

plt.ylabel("Values")
```

```
plt.grid(axis='y', linestyle='--', alpha=0.5)

plt.tight_layout()

plt.show()
```



Summary:

1) The average fraudulent transaction amount is very high.

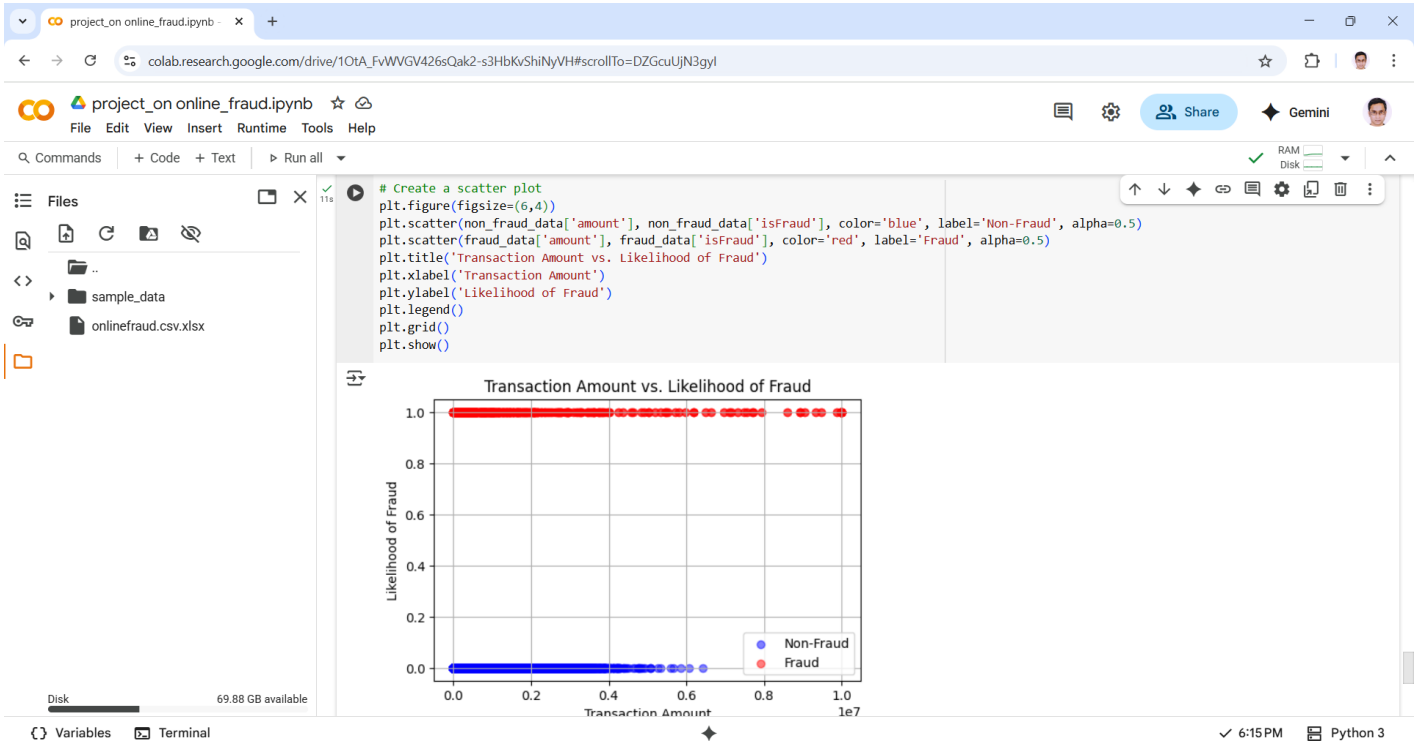
2) There are only 1,132 unique destination accounts involved in fraudulent transactions, which implies that fraud may be organized and **targeting specific shell accounts** repeatedly. Because a small number of accounts is being used repeatedly, they could either be blacklisted or closely monitored for fraud detection.

8) **Correlation Analysis**: What is the correlation between transaction amount and the likelihood of fraud? Can we create a scatter plot to visualize this relationship?

Investigating the correlation between transaction amounts and fraud likelihood can reveal critical insights into risk factors. A scatter plot can visually represent this relationship, aiding in the development of more sophisticated fraud detection algorithms.

How it was done:

```
# Create a scatter plot

plt.figure(figsize=(6,4))

plt.scatter(non_fraud_data['amount'], non_fraud_data['isFraud'], color='blue', label='Non-Fraud', alpha=0.5)

plt.scatter(fraud_data['amount'], fraud_data['isFraud'], color='red', label='Fraud', alpha=0.5)

plt.title('Transaction Amount vs. Likelihood of Fraud')

plt.xlabel('Transaction Amount')

plt.ylabel('Likelihood of Fraud')

plt.legend()

plt.grid()

plt.show()
```



Summary:

1) Most of the red points (fraud) are clustered at the higher end of the transaction amount scale suggesting that fraud is significantly more likely in high-value transactions.

2) Blue dots (non-fraud) are spread across all ranges of transaction amounts, including low, medium, and high suggesting that low and mid-range transactions are generally legitimate, with less risk.

9) Flagged Fraud Analysis: What percentage of flagged transactions are actually fraudulent? How does this compare to non-flagged transactions?

Understanding the effectiveness of flagged transactions in identifying actual fraud is essential for refining detection systems. This analysis can help improve the accuracy of fraud alerts and reduce false positives.

How it was done:

```
# Total flagged transactions

total_flagged = df[df['isFlaggedFraud'] == 1]

# Fraudulent flagged transactions

fraud_flagged = total_flagged[total_flagged['isFraud'] == 1]

# Total non-flagged transactions

total_nonflagged = df[df['isFlaggedFraud'] == 0]

# Fraudulent non-flagged transactions

fraud_nonflagged = total_nonflagged[total_nonflagged['isFraud'] == 1]


# Percentage calculations

percent_fraud_in_flagged = (len(fraud_flagged) / len(total_flagged)) * 100 if len(total_flagged) > 0 else 0

percent_fraud_in_nonflagged = (len(fraud_nonflagged) / len(total_nonflagged)) * 100

print(f"Flagged Transactions: {len(total_flagged)}")

print(f"Fraudulent Flagged Transactions: {len(fraud_flagged)} → {percent_fraud_in_flagged:.4f}%")

print(f"Non-Flagged Transactions: {len(total_nonflagged)}")

print(f"Fraudulent Non-Flagged Transactions: {len(fraud_nonflagged)} → {percent_fraud_in_nonflagged:.4f}%")
```

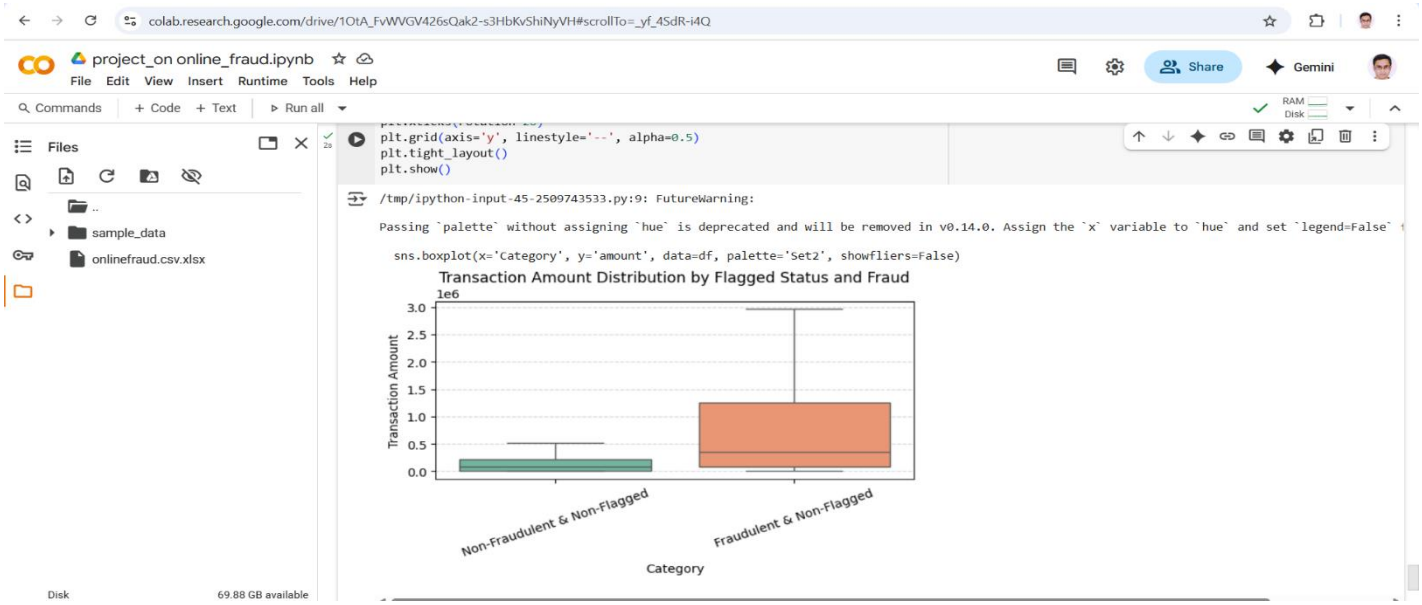
Flagged Transactions: 0

Fraudulent Flagged Transactions: 0 → 0.0000%

Non-Flagged Transactions: 1048575

Fraudulent Non-Flagged Transactions: 1142 → 0.1089%

```
# Create a 'Category' column based on 'isFraud' and 'isFlaggedFraud'
df['Category'] = 'Non-Fraudulent & Non-Flagged'
df.loc[(df['isFraud'] == 1) & (df['isFlaggedFraud'] == 0), 'Category'] = 'Fraudulent & Non-Flagged'
df.loc[(df['isFraud'] == 0) & (df['isFlaggedFraud'] == 1), 'Category'] = 'Non-Fraudulent & Flagged'
df.loc[(df['isFraud'] == 1) & (df['isFlaggedFraud'] == 1), 'Category'] = 'Fraudulent & Flagged'
# Plotting box plot
plt.figure(figsize=(6,4))
sns.boxplot(x='Category', y='amount', data=df, palette='Set2', showfliers=False)
plt.title("Transaction Amount Distribution by Flagged Status and Fraud")
plt.ylabel("Transaction Amount")
plt.xticks(rotation=20)
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



Summary:

1) The median transaction amount in the fraudulent & non-flagged group is significantly higher than in the non-fraudulent group indicating that fraudulent transactions tend to involve higher monetary amounts.

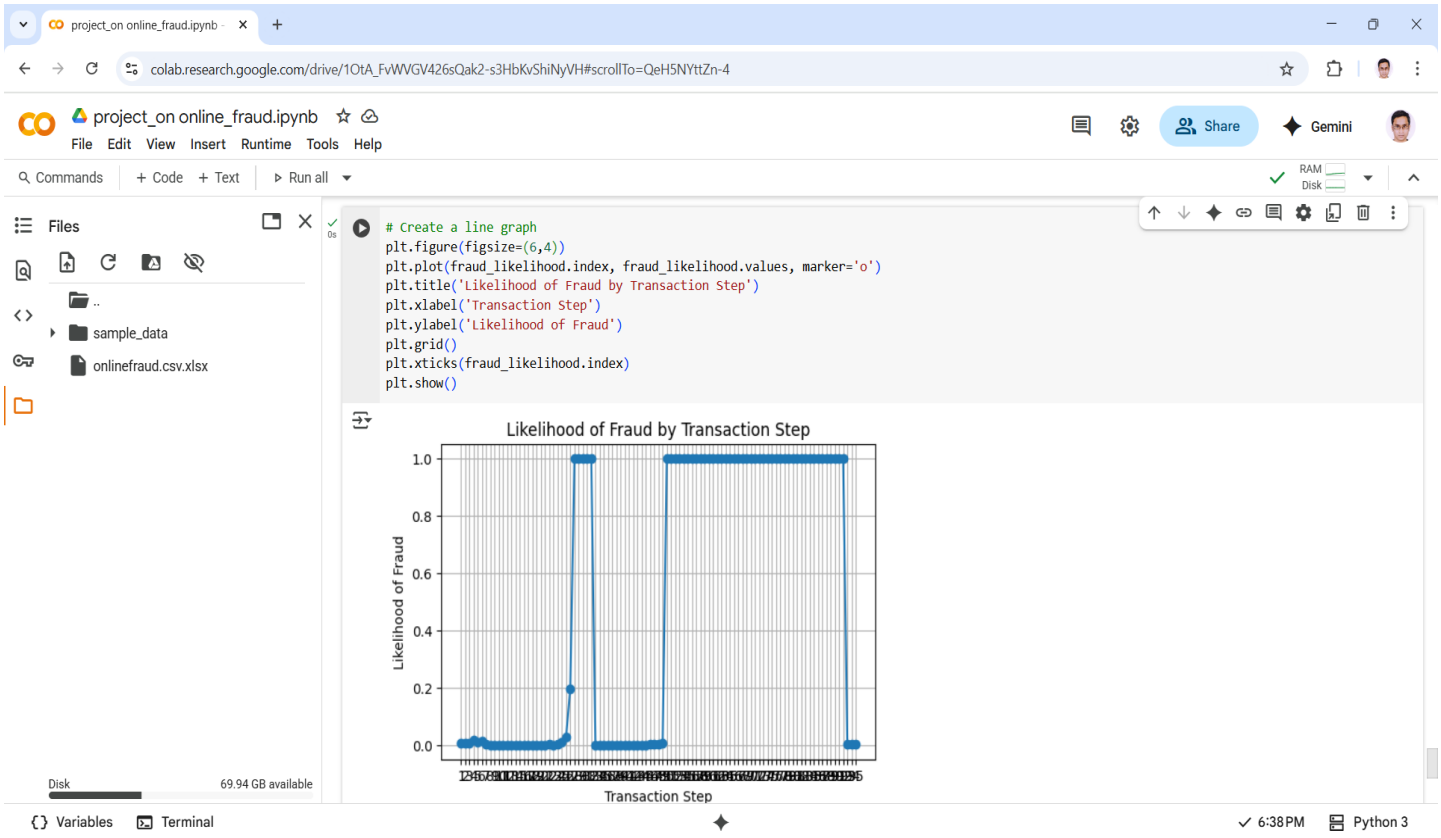
2) The **interquartile range** for fraudulent transactions is much larger, indicating **greater variability** in transaction amounts suggesting fraud is not limited to just high-value transactions—some may still be small, but there's a **higher risk** with larger amounts.

10) Impact of Transaction Steps: How does the step number in the transaction process relate to the likelihood of fraud? Can we create a line graph to show this relationship?

Understanding the effectiveness of flagged transactions in identifying actual fraud is essential for refining detection systems. This analysis can help improve the accuracy of fraud alerts and reduce false positives.

How it was done

```
# Group by step and calculate the mean likelihood of fraud

fraud_likelihood = df.groupby('step')['isFraud'].mean()

# Create a line graph

plt.figure(figsize=(6,4))

plt.plot(fraud_likelihood.index, fraud_likelihood.values, marker='o')

plt.title('Likelihood of Fraud by Transaction Step')

plt.xlabel('Transaction Step')

plt.ylabel('Likelihood of Fraud')

plt.grid()

plt.xticks(fraud_likelihood.index)

plt.show()
```



Summary:

1) There are long sequences of steps where fraud does not occur at all which indicates that frauds are targeted, intermittent, or timed with precision rather than happening randomly or continuously.

Key Inferences from the project:

1) Fraud is rare, out of all the transactions only 0.109% transactions are fraudulent, but there is financial damage from these cases which makes their detection to be important.

2) Fraudulent transactions tend to involve significantly higher amounts, highlighting that fraudsters aim to gain maximum per transaction attempt.

3) Fraud is concentrated in only two types: Cash Out and Transfer.

4) Fraud isn't happening uniformly, showing that fraudsters operate during specific time windows.

5) A clear positive relationship was observed between high transaction amounts and the probability of fraud.

6) Visual tools such as box plots, bar charts, scatter plots, and time series helped uncover critical fraud patterns, improving interpretability and supporting effective feature engineering for detection models.

Fraud Prevention Measures Based on Project Findings

1) Since **fraud is only observed in CASH_OUT and TRANSFER types**, stricter rules, verification, or OTPs for these transaction types could be implemented. Real-time monitoring only for these types to reduce processing overhead.

2) As Fraud is strongly correlated with large transaction amounts, thresh-hold based alerts could be set, like temporary holding of higher transactions for manual review or identity revaluation.

3) As seen in the pattern that fraud tends to spike in specific step intervals (time windows), a time-window anomaly detection could be used to detect fraud bursts in real-time.

4) Since very few destination accounts (~0.09%) were involved in most frauds, a watchlist could be maintained of those high-risk destination accounts and can be flagged under fraudulent or even black-listed under preventive measures.